

雲端法律專家：運用 Google (合法) 駭入法院系統

來源：<https://codelabs.developers.google.com/legal-eagle-rag/instructions?hl=zh-tw>

步驟數：14

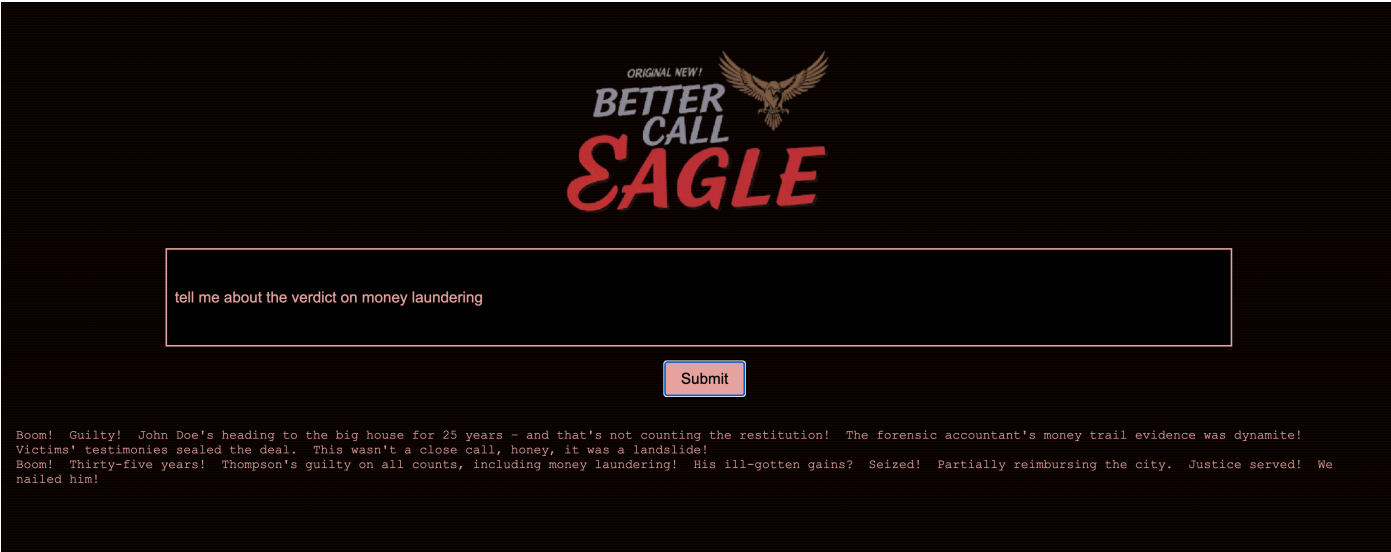
本工作坊將引導您使用 Google Cloud AI 工具建構法律助理。強調如何處理、瞭解及搜尋非結構化資料。您將學習如何使用檢索增強生成 (RAG) 方法，根據實際法律資訊提供更準確實用的回覆。

目錄

1. [1. 簡介](#)
2. [2. 架構](#)
3. [3. 事前準備](#)
4. [4. 使用 Gemini Code Assist 撰寫推論應用程式](#)
5. [5. 在 Cloud Editor 中進行本機測試](#)
6. [6. 設定向量儲存庫](#)
7. [7. 將資料載入向量儲存區](#)
8. [8. 設定 Cloud Run 函式](#)
9. [9. 測試及建構 Cloud Run 函式](#)
10. [10. 建構容器映像檔並推送至 Artifacts 存放區](#)
11. [11. 建立 Cloud Run 函式並設定 Eventarc 觸發條件](#)
12. [12. 將法律文件上傳至 GCS bucket](#)
13. [13. 導入 RAG](#)
14. [14. 挑戰](#)

1. 簡介

我一直很著迷於法庭的緊張氛圍，想像自己能巧妙地應對複雜情況，並做出強而有力的結辯。雖然我的職涯發展方向不同，但很高興能告訴大家，在 AI 的協助下，我們或許都能更接近實現法庭夢想。



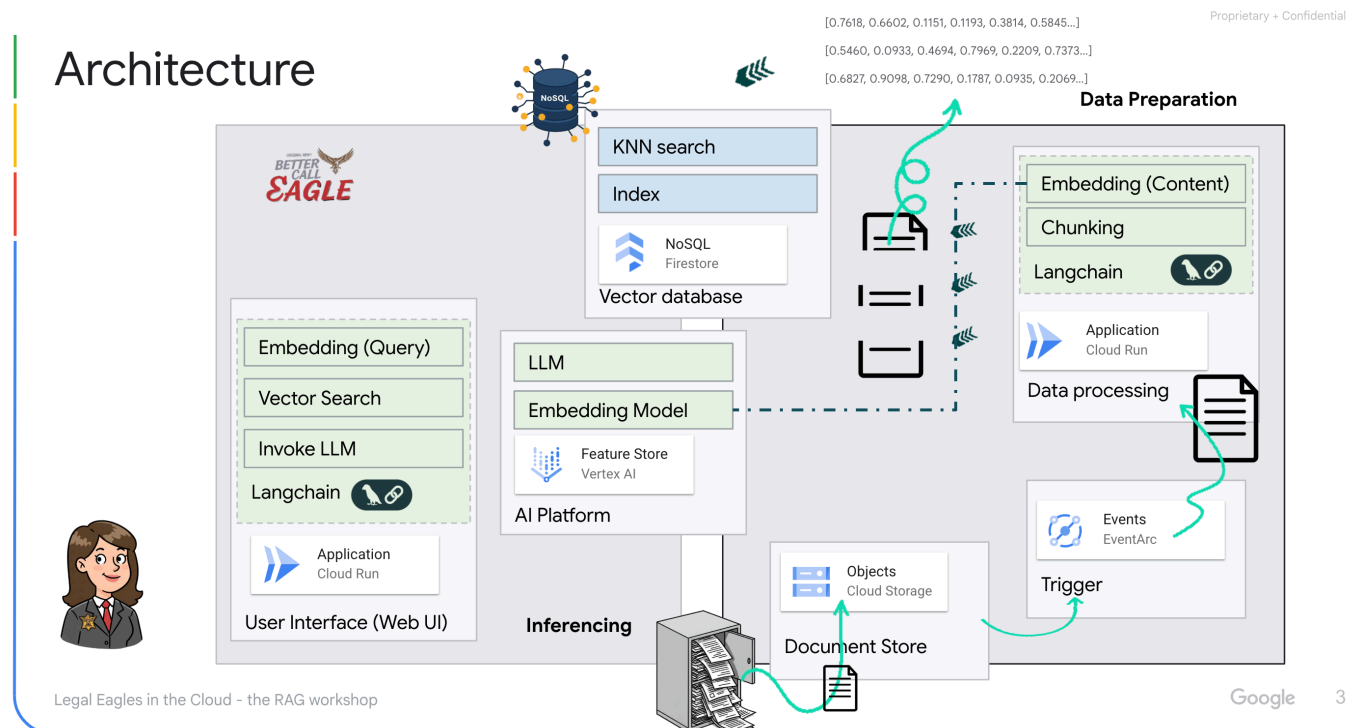
今天，我們要深入探討如何使用 Google 強大的 AI 工具 (例如 Vertex AI、Firestore 和 Cloud Run Functions) 處理及解讀法律資料、執行快速搜尋，以及協助虛構客戶 (或您自己) 擺脫困境。

您可能不會對證人進行交互訊問，但透過我們的系統，您將能從大量資訊中篩選出最相關的資料，並在幾秒內生成清楚的摘要。

2. 架構

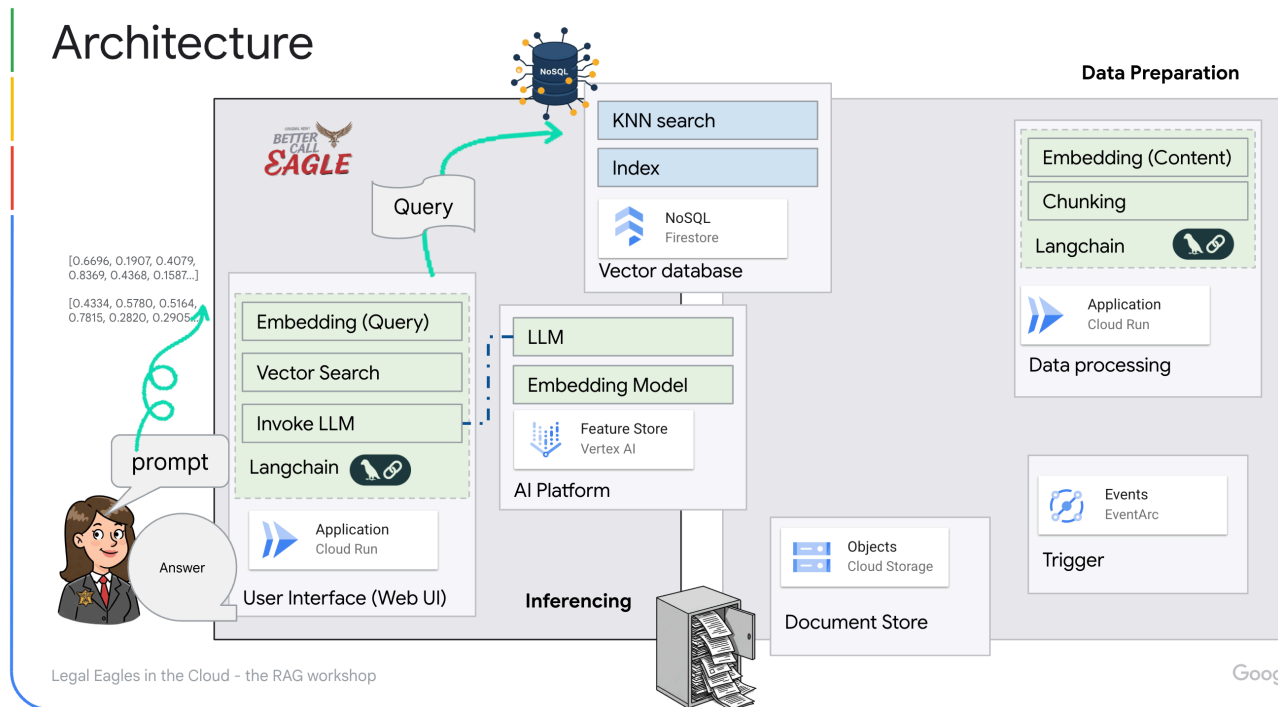
本專案著重於使用 Google Cloud AI 工具建構法律助理，並強調如何處理、解讀及搜尋法律資料。這套系統可篩選大量資訊、生成摘要，並快速呈現相關資料。法律助理的架構包含幾個重要元件：

從非結構化資料建構知識庫：Google Cloud Storage (GCS) 用於儲存法律文件。Firestore 是 NoSQL 資料庫，可做為向量儲存空間，存放文件區塊和對應的嵌入內容。Firestore 已啟用 Vector Search，可進行相似度搜尋。當新的法律文件上傳至 GCS 時，Eventarc 會觸發 Cloud Run 函式。這個函式會將文件分割成多個區塊，並使用 Vertex AI 的文字嵌入模型，為每個區塊生成嵌入。這些嵌入內容隨後會與文字區塊一併儲存在 Firestore 中。



採用 LLM 和 RAG 技術的應用程式：問答系統的核心是 `ask_llm` 函式，這個函式會使用 langchain 程式庫與 Vertex AI Gemini 大型語言模型互動。這會根據使用者的查詢建立 `HumanMessage`，並納入 `SystemMessage`，指示 LLM 扮演實用的法律助理。系統會採用檢索增強生成 (RAG) 方法，在回答查詢前，先使用 `search_resource` 函式從 Firestore 向量儲存庫擷取相關脈絡。接著，系統會將這個脈絡納入 `SystemMessage`，讓 LLM 的回覆以提供的法律資訊為依據。

Architecture



這項專案的目標是透過 RAG 擺脫 LLM 的「創意詮釋」，先從可信的法律來源擷取相關資訊，再生成答案。因此能根據實際法律資訊，提供更準確、更具參考價值的回覆。這個系統是使用各種 Google Cloud 服務建構而成，例如 Google Cloud Shell、Vertex AI、Firestore、Cloud Run 和 Eventarc。

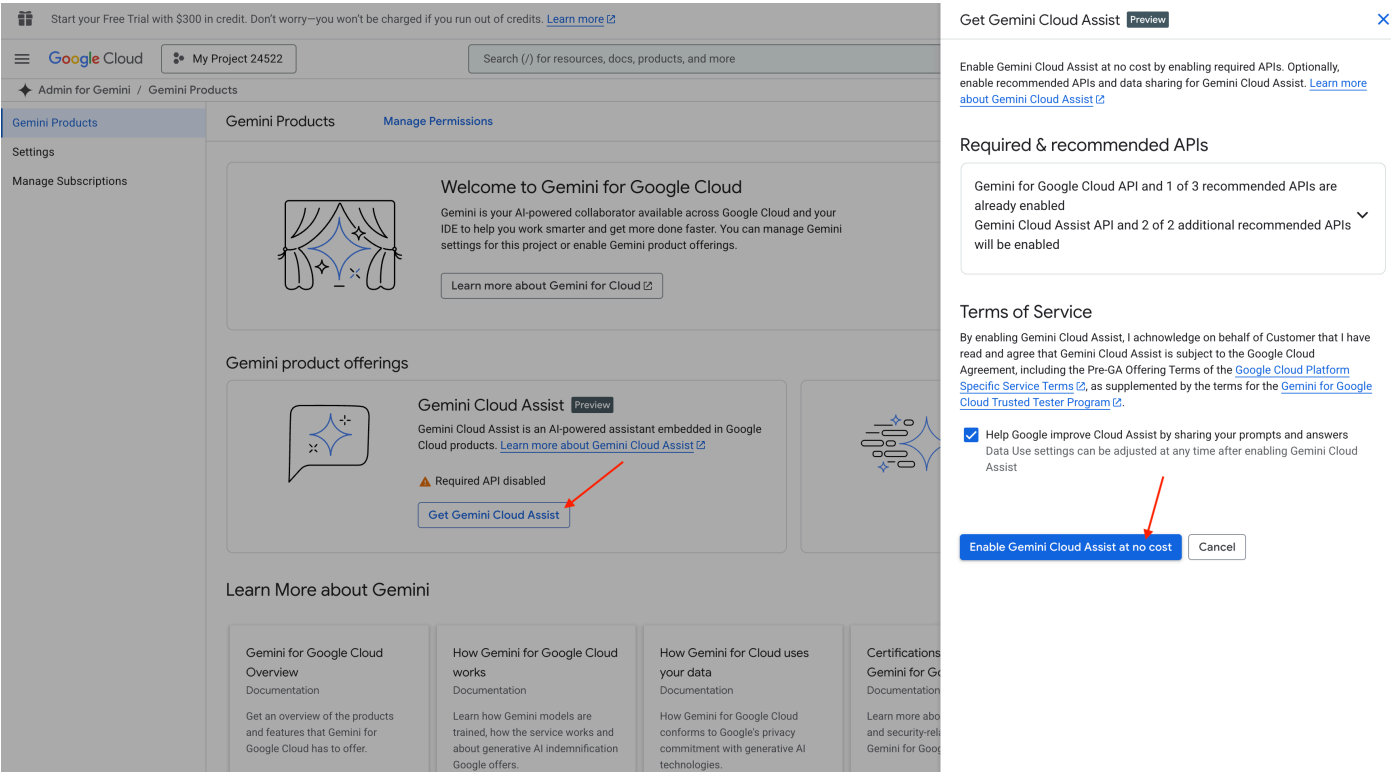
步驟 3 · 約 0 分鐘

3. 事前準備

在 [Google Cloud 控制台](#) 的專案選取器頁面中，選取或建立 Google Cloud 專案。確認 Cloud 專案已啟用計費功能。[瞭解如何檢查專案是否已啟用計費功能](#)。

在 Cloud Shell IDE 中啟用 Gemini Code Assist

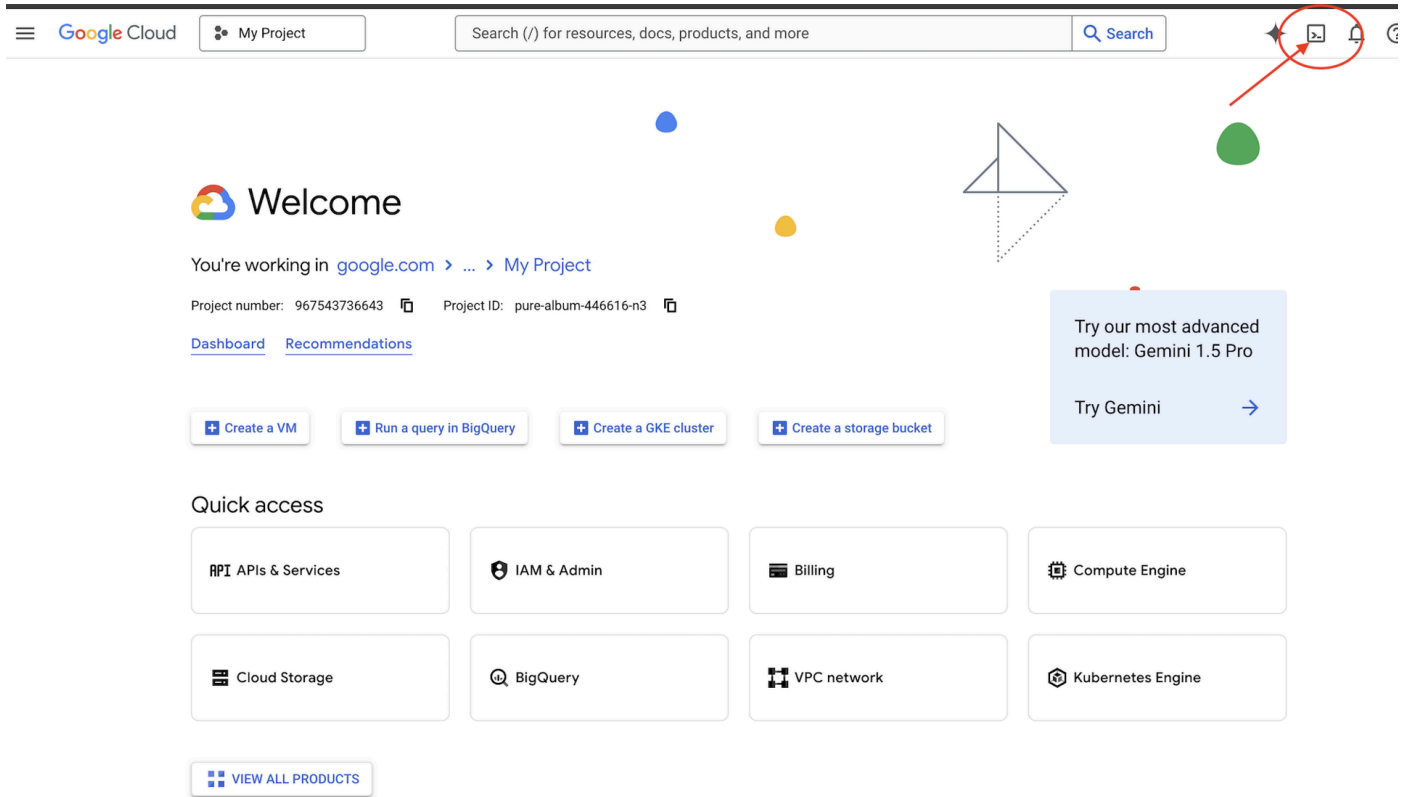
👉 在 Google Cloud 控制台中，前往 [Gemini Code Assist Tools](#)，同意條款及細則即可免費啟用 Gemini Code Assist。



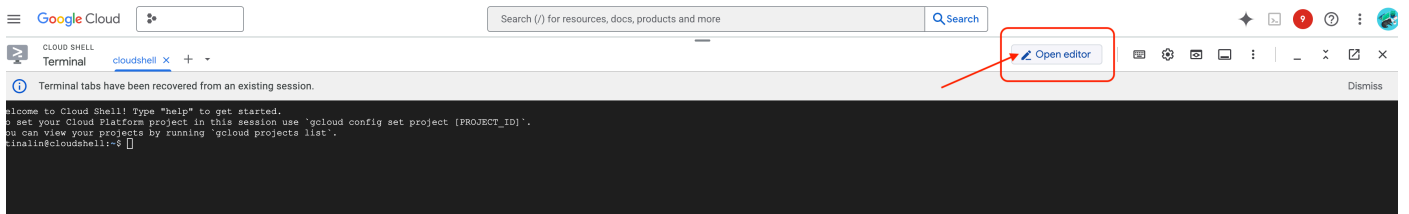
忽略權限設定，離開這個頁面。

使用 Cloud Shell 編輯器

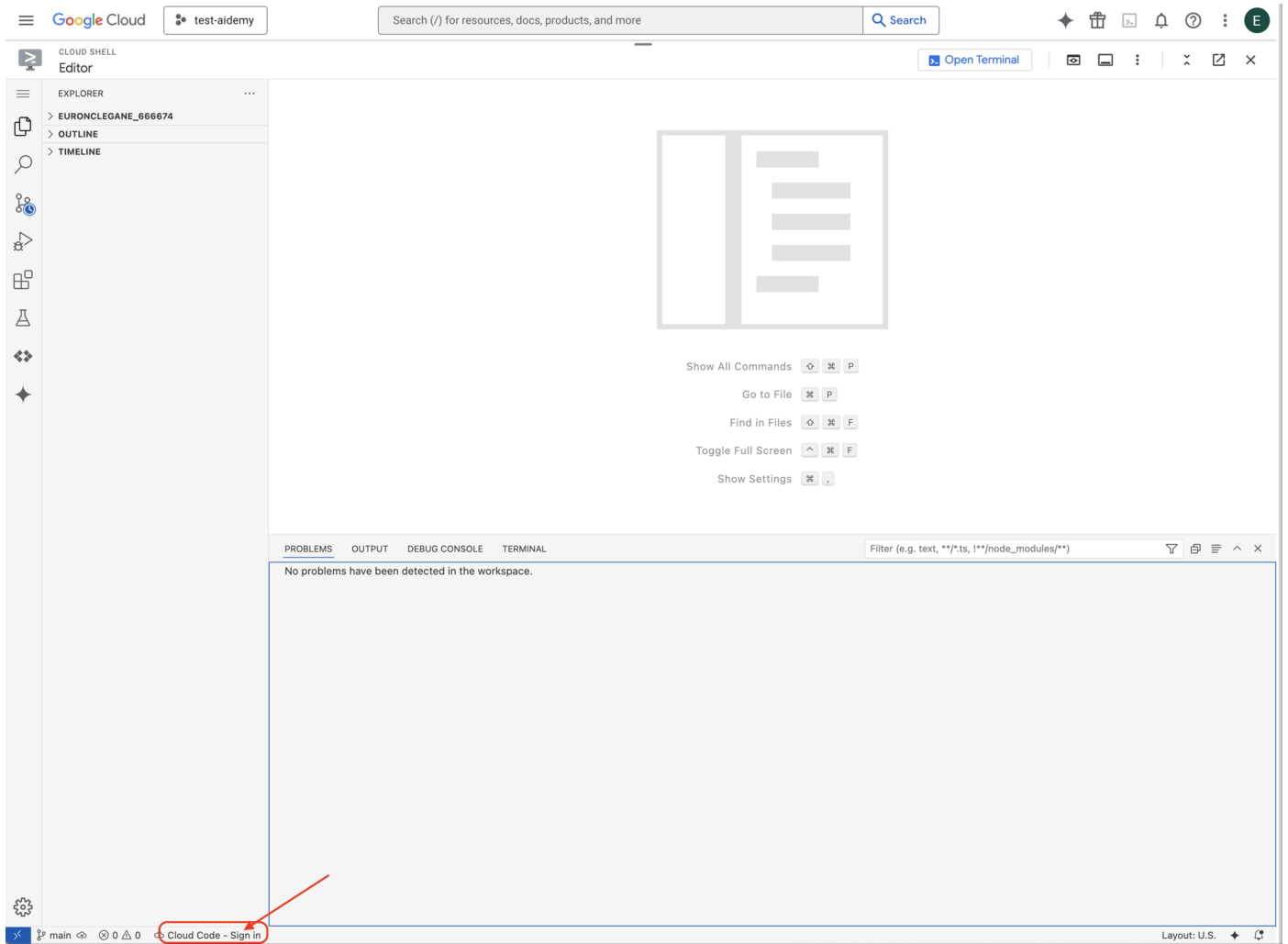
👉 點選 Google Cloud 控制台頂端的「啟用 Cloud Shell」(這是 Cloud Shell 窗格頂端的終端機形狀圖示)



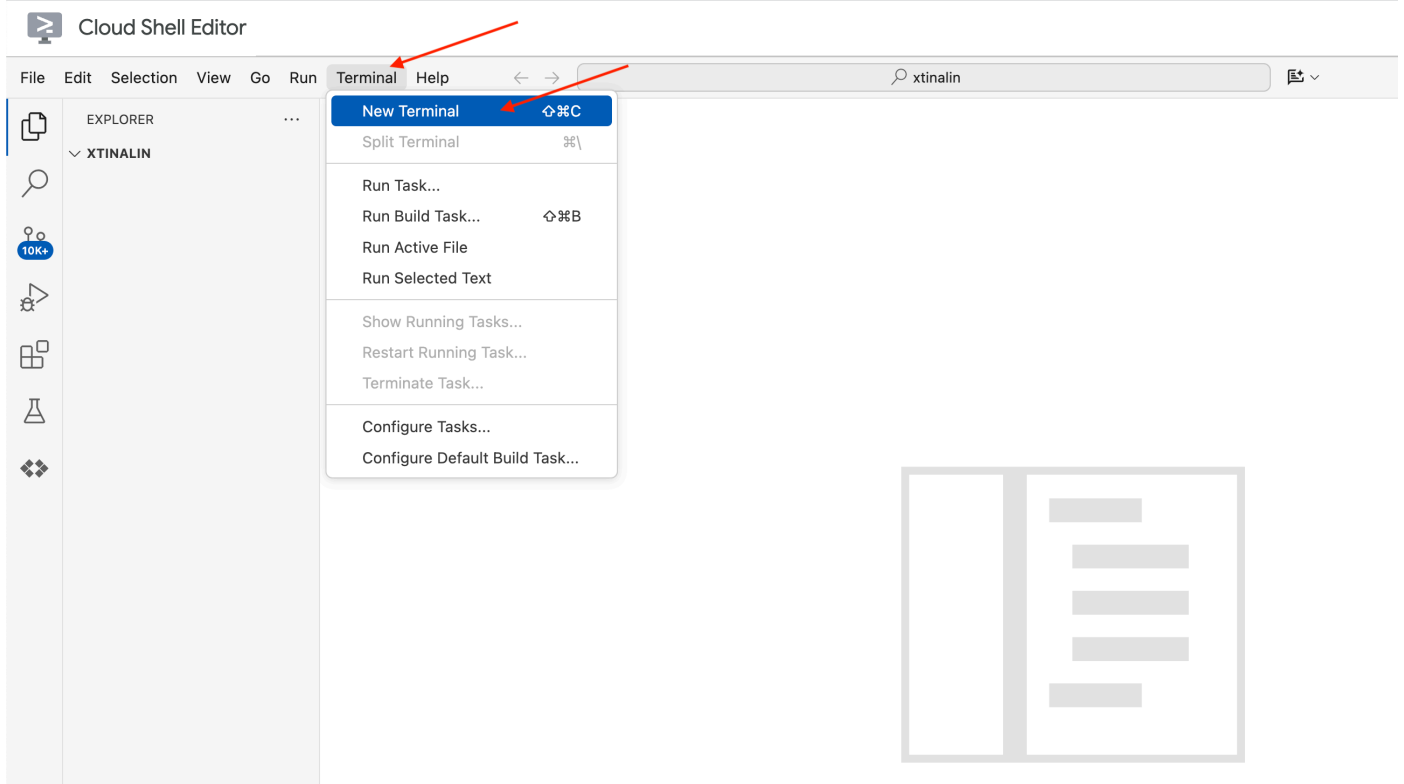
✎ 按一下「Open Editor」按鈕 (類似於有鉛筆的開啟資料夾)。這會在視窗中開啟 Cloud Shell 編輯器。左側會顯示檔案總管。



✎ 點選底部狀態列中的「Cloud Code Sign-in」按鈕，如下圖所示。依指示授權外掛程式。如果狀態列顯示「Cloud Code - no project」，請選取該項目，然後在「Select a Google Cloud Project」(選取 Google Cloud 專案) 下拉式選單中，選取您打算使用的特定 Google Cloud 專案。



👉 在雲端 IDE 中開啟終端機



👉 在新終端機中，使用下列指令驗證您是否已通過驗證，以及專案是否已設為您的專案 ID：

```
gcloud auth list
```

👉 點選 Google Cloud 控制台頂端的「啟用 Cloud Shell」。

```
gcloud config set project <YOUR_PROJECT_ID>
```

👉 執行下列指令，啟用必要的 Google Cloud API：

```
gcloud services enable storage.googleapis.com \
                        run.googleapis.com \
                        artifactregistry.googleapis.com \
                        aiplatform.googleapis.com \
                        eventarc.googleapis.com \
                        cloudresourcemanager.googleapis.com \
                        firestore.googleapis.com \
                        clouddaicompanion.googleapis.com
```

在 Cloud Shell 工具列 (位於 Cloud Shell 窗格頂端)，按一下「Open Editor」(開啟編輯器) 按鈕 (看起來像是有鉛筆的開啟資料夾)。視窗中會開啟 Cloud Shell 程式碼編輯器。左側會顯示檔案總管。

👉 在終端機下載 Bootstrap Skeleton Project：

```
git clone https://github.com/weimeilin79/legal-eagle.git
```

選用：西班牙文版

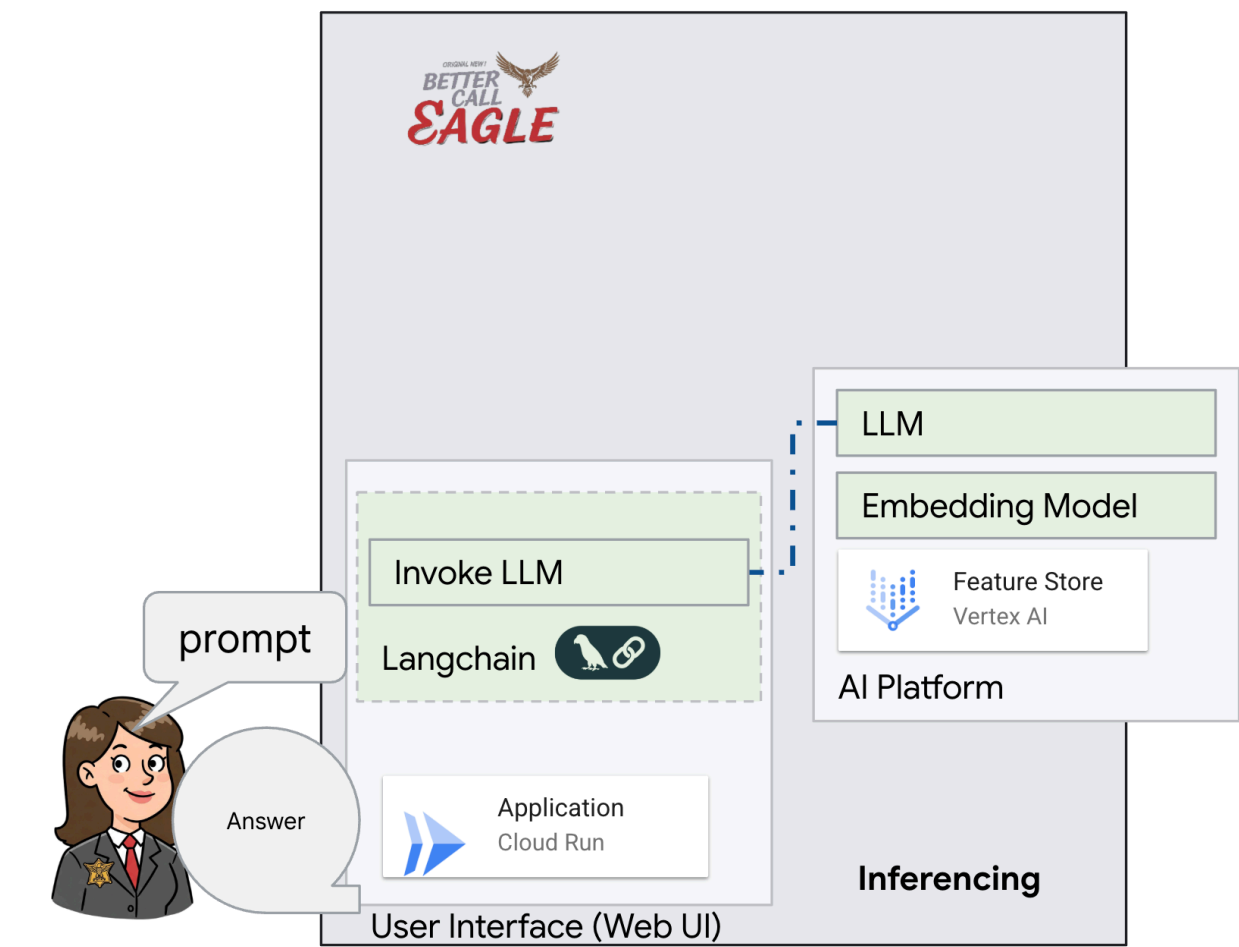
👉 Existe una versión alternativa en español. Por favor, utilice la siguiente instrucción para clonar la versión correcta.

```
git clone -b spanish https://github.com/weimeilin79/legal-eagle.git
```

在 Cloud Shell 終端機中執行這項指令後，Cloud Shell 殼層環境中會建立名為 `legal-eagle` 的新資料夾。

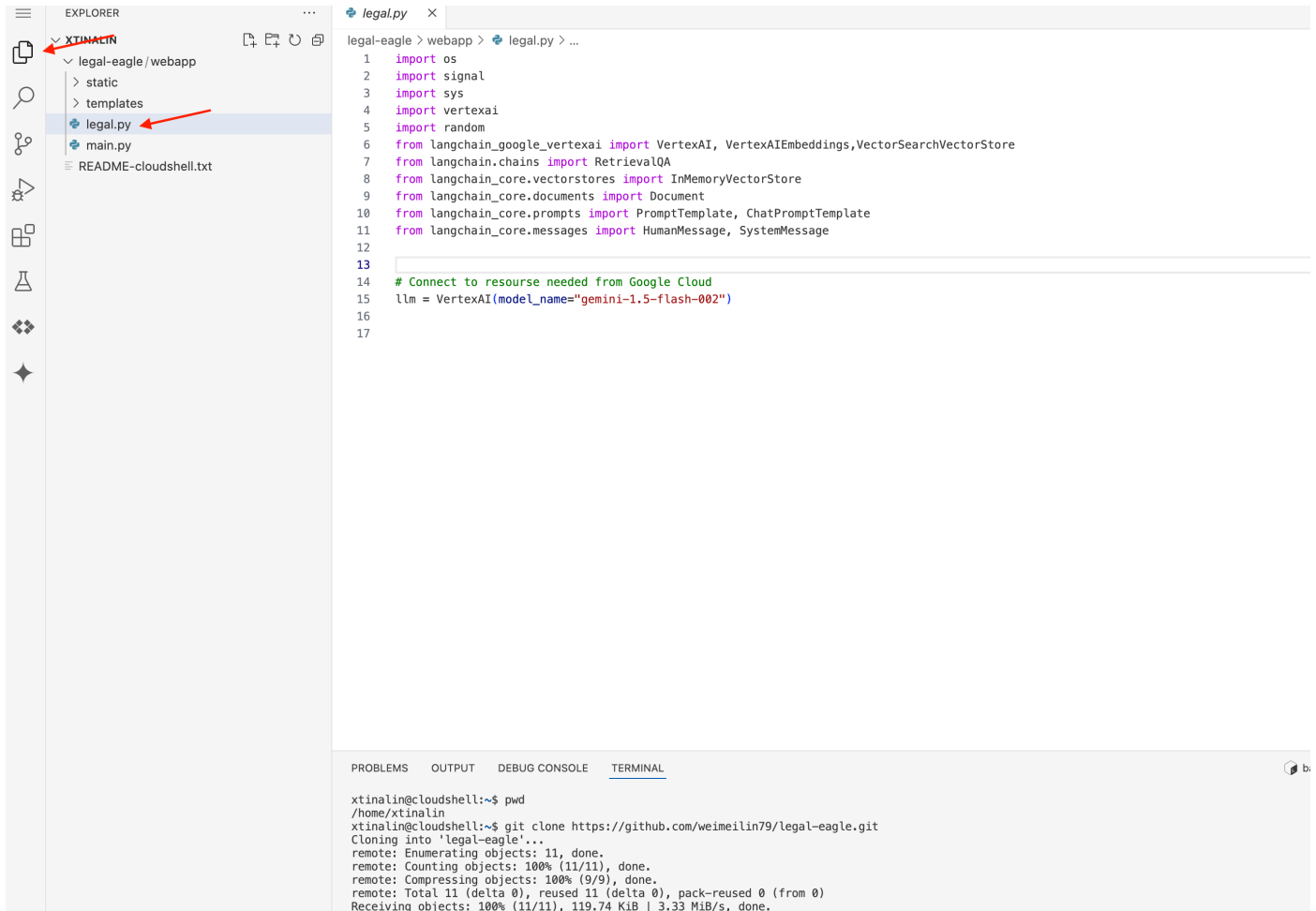
4. 使用 Gemini Code Assist 撰寫推論應用程式

在本節中，我們將著重於建構法律助理的核心部分，也就是接收使用者問題並與 AI 模型互動以生成答案的網頁應用程式。我們會運用 Gemini Code Assist 協助編寫這部分推論的 Python 程式碼。



一開始，我們會建立 Flask 應用程式，使用 LangChain 程式庫直接與 Vertex AI Gemini 模型通訊。第一版會根據模型的一般知識，提供實用的法律助理服務，但還無法存取特定法院案件文件。這樣我們就能在稍後使用 RAG 強化 LLM 之前，先瞭解 LLM 的基準效能。

在 Cloud Code Editor 的「Explorer」窗格 (通常位於左側) 中，您現在應該會看到複製 Git 存放區時建立的資料夾 `legal-eagle` 在「Explorer」中開啟專案的根資料夾。你會在其中看到 `webapp` 子資料夾，請一併開啟。□



👉 在 Cloud Code 編輯器中編輯 `legal.py` 檔案，你可以使用不同方法提示 Gemini Code Assist。

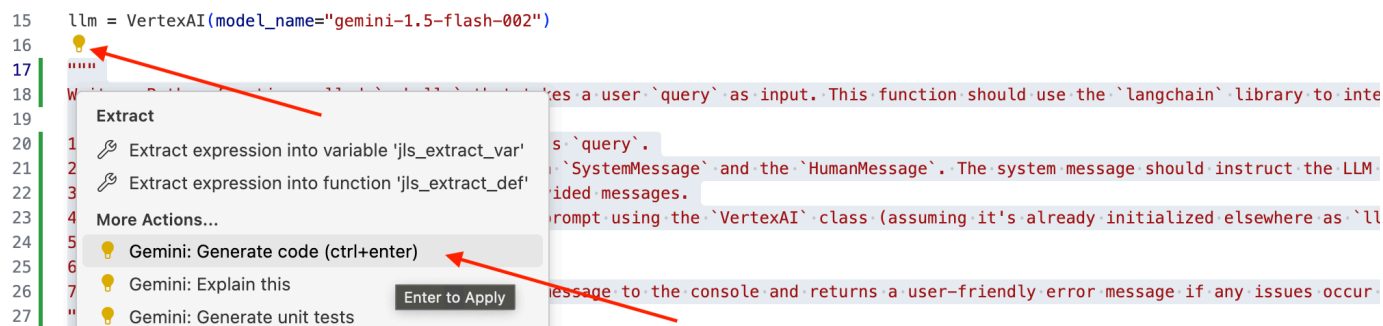
👉 將下列提示詞複製到 `legal.py` 底部，清楚說明要讓 Gemini Code Assist 生成的內容，然後點按顯示的燈泡💡圖示，並選取「Gemini: Generate Code」（實際選單項目可能因 Cloud Code 版本而略有不同）。

```
"""
```

Write a Python function called `ask_llm` that takes a user `query` as input. This function should use the `langchain` library to interact with a Vertex AI Gemini Large Language Model. Specifically, it should:

1. Create a `HumanMessage` object from the user's `query`.
2. Create a `ChatPromptTemplate` that includes a `SystemMessage` and the `HumanMessage`. The system message should instruct the LLM to act as a helpful assistant in a courtroom setting, aiding an attorney by providing necessary information. It should also specify that the LLM should respond in a high-energy tone, using no more than 100 words, and offer a humorous apology if it doesn't know the answer.
3. Format the `ChatPromptTemplate` with the provided messages.
4. Invoke the Vertex AI LLM with the formatted prompt using the `VertexAI` class (assuming it's already initialized elsewhere as `llm`).
5. Print the LLM's `response`.
6. Return the `response`.
7. Include error handling that prints an error message to the console and returns a user-friendly error message if any issues occur during the process. The Vertex AI model should be "gemini-2.0-flash".

```
"""
```



仔細檢查生成的程式碼

- 是否大致按照你在註解中列出的步驟操作？
- 是否會使用 `SystemMessage` 和 `HumanMessage` 建立 `ChatPromptTemplate`？
- 是否包含基本錯誤處理 (`try...except`)？

如果生成的程式碼良好且大致正確，您可以接受 (按下 Tab 鍵或 Enter 鍵可接受內嵌建議，或按一下「接受」可接受較大的程式碼區塊)。

如果生成的程式碼不完全符合需求或有錯誤，請別擔心！**Gemini Code Assist** 是一項輔助工具，而非在第一次嘗試時就編寫出完美程式碼。

編輯及修改生成的程式碼，以進行修正、更正錯誤，並更符合您的需求。在 Code Assist 對話面板中新增更多註解或提出特定問題，進一步提示 **Gemini Code Assist**。

如果您是 SDK 新手，可以參考這個運作範例。

👉 複製並貼上下列程式碼，然後「取代」 `legal.py` 中的程式碼：

```

import os
import signal
import sys
import vertexai
import random
from langchain_google_vertexai import VertexAI
from langchain_core.prompts import PromptTemplate, ChatPromptTemplate
from langchain_core.messages import HumanMessage, SystemMessage
# Connect to resource needed from Google Cloud
llm = VertexAI(model_name="gemini-2.0-flash")
def ask_llm(query):
    try:
        query_message = {
            "type": "text",
            "text": query,
        }

        input_msg = HumanMessage(content=[query_message])
        prompt_template = ChatPromptTemplate.from_messages(
            [
                SystemMessage(
                    content=(
                        "You are a helpful assistant, and you are with the attorney in a
courtroom, you are helping him to win the case by providing the information he needs "
                        "Don't answer if you don't know the answer, just say sorry in a funny
way possible"
                        "Use high energy tone, don't use more than 100 words to answer"
                        # f"Here is some past conversation history between you and the user
{relevant_history}"
                        # f"Here is some context that is relevant to the question
{relevant_resource} that you might use"
                    )
                ),
                input_msg,
            ]
        )
        prompt = prompt_template.format()
        response = llm.invoke(prompt)
        print(f"response: {response}")
        return response
    except Exception as e:
        print(f"Error sending message to chatbot: {e}") # Log this error too!
        return f"Unable to process your request at this time. Due to the following reason:
{str(e)}"

```

👉 選用：西班牙文版

Sustituye el siguiente texto como se indica: `You are a helpful assistant, to` `You are a helpful assistant that speaks Spanish,`

接著，建立函式來處理路由，以回覆使用者的問題。

在 Cloud Shell 編輯器中開啟 `main.py`。與您在 `legal.py` 中生成 `ask_llm` 的方式類似，使用 **Gemini Code Assist** 生成 *Flask* 路由和 `ask_question` 函式。在 `main.py` 中輸入下列 PROMPT 做為註解 (請務必在 `if __name__ == "__main__":` 啟動 Flask 應用程式之前新增)：

```
.....
@app.route('/', methods=[ 'GET' ])
def index():
    return render_template('index.html')

"""
PROMPT:
Create a Flask endpoint that accepts POST requests at the '/ask' route.
The request should contain a JSON payload with a 'question' field. Extract the question from
the JSON payload.
Call a function named ask_llm (located in a file named legal.py) with the extracted question
as an argument.
Return the result of the ask_llm function as the response with a 200 status code.
If any error occurs during the process, return a 500 status code with the error message.
"""

# Add this block to start the Flask app when running locally
if __name__ == "__main__":
    .....
```

只有在生成的程式碼良好且大致正確時，才接受程式碼。如果您不熟悉 Python，請複製並貼上這個範例到 `main.py` 中，現有程式碼下方。

👉 請務必在網頁應用程式啟動前貼上以下內容 (如果名稱為「main」：)

```
@app.route('/ask', methods=[ 'POST' ])
def ask_question():
    data = request.get_json()
    question = data.get('question')
    try:
        # call the ask_llm in legal.py
        answer_markdown = legal.ask_llm(question)

        print(f"answer_markdown: {answer_markdown}")
        # Return the Markdown as the response
        return answer_markdown, 200
    except Exception as e:
        return f"Error: {str(e)}", 500 # Handle errors appropriately
```

按照這些步驟操作，您應該就能順利啟用 **Gemini Code Assist**、設定專案，並使用這項工具在 `main.py` 檔案中生成 `ask` 函式。

步驟 5 · 約 0 分鐘

5. 在 Cloud Editor 中進行本機測試

👉 在編輯器的終端機中，安裝相依程式庫並在本機啟動網頁 UI。

```
cd ~/legal-eagle/webapp
python -m venv env
source env/bin/activate
export PROJECT_ID=$(gcloud config get project)
pip install -r requirements.txt
python main.py
```

在 Cloud Shell 終端機輸出內容中尋找啟動訊息。Flask 通常會列印訊息，指出正在執行，以及執行所在的通訊埠。

- Running on http://127.0.0.1:8080

應用程式必須持續執行，才能處理要求。

👉 從「網頁預覽」選單中，選擇「透過以下通訊埠預覽：8080」。Cloud Shell 會開啟新的瀏覽器分頁或視窗，顯示應用程式的網頁預覽畫面。



👉 在應用程式介面中，輸入幾個與法律案件參考資料相關的問題，看看 LLM 的回覆。舉例來說，您可以嘗試：

- Michael Brown 被判處幾年有期徒刑？
- 陳小珍的動作導致多少未授權費用？
- 在 Emily White 案件的調查中，鄰居的證詞扮演什麼角色？

👉 選用：西班牙文版

- ¿A cuántos años de prisión fue sentenciado Michael Brown?
- ¿Cuánto dinero en cargos no autorizados se generó como resultado de las acciones de Jane Smith?
- ¿Qué papel jugaron los testimonios de los vecinos en la investigación del caso de Emily White?

仔細查看答案後，您可能會發現模型有時會產生幻覺、含糊不清或籠統的答案，有時還會誤解您的問題，尤其模型目前還無法存取特定法律文件。

👉 請按下 **Ctrl+C** 停止指令碼。

👉 退出虛擬環境，在終端機中執行：

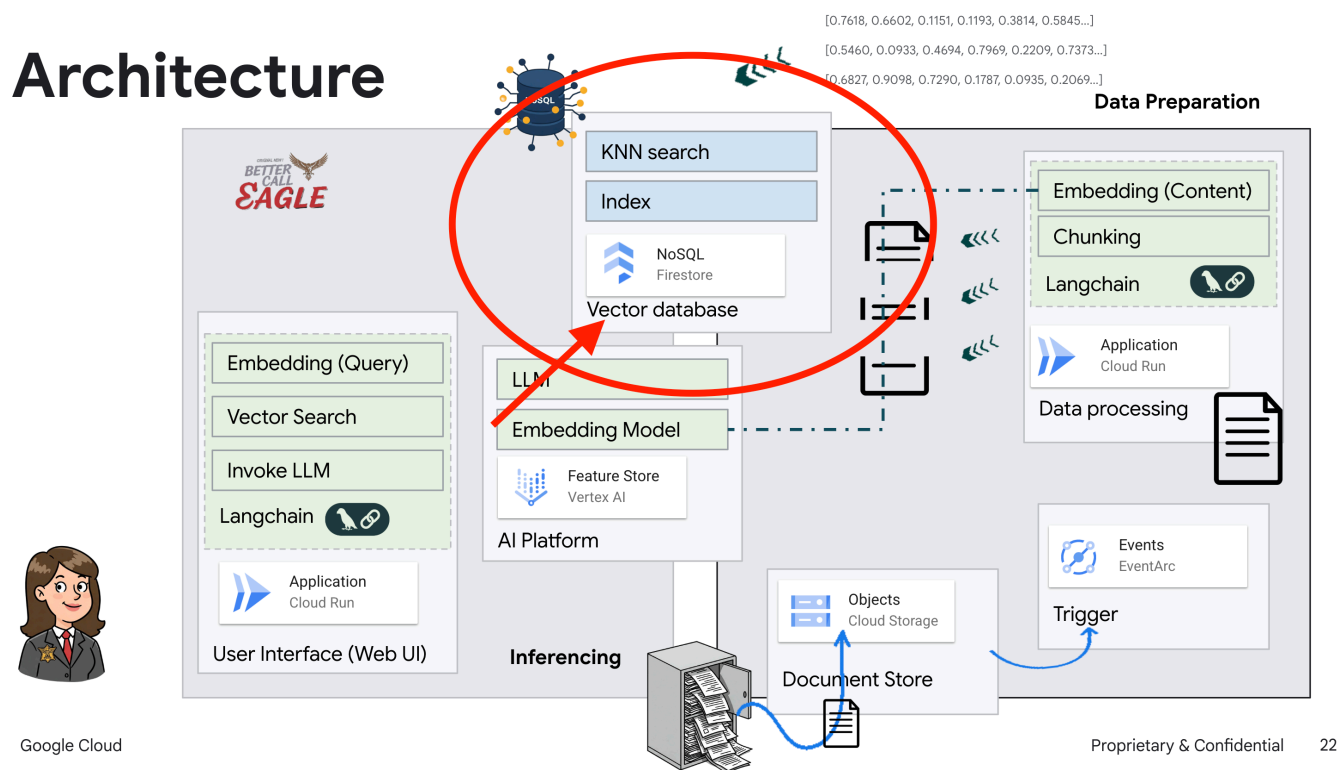
deactivate

6. 設定向量儲存庫

是時候終結 LLM 對法律的「創意解讀」了。這時，檢索增強生成 (RAG) 技術就能派上用場！這就像是在 LLM 回答問題前，先讓它存取功能強大的法律圖書館。RAG 不會單純依賴一般知識 (這類知識可能模糊不清或過時，視模型而定)，而是先從可信來源 (在本例中為法律文件) 擷取相關資訊，然後根據這些資訊生成更準確的回覆。這就像 LLM 在進入法庭前先做功課！

如要建構 RAG 系統，我們需要儲存所有法律文件的位置，更重要的是，這些文件必須可依意義搜尋。這時 Firestore 就能派上用場！Firestore 是 Google Cloud 彈性且可擴充的 NoSQL 文件資料庫。

我們將使用 **Firestore** 做為我們的 *向量儲存庫*。我們會將法律文件分塊儲存在 **Firestore** 中，並為每個分塊儲存其嵌入內容，也就是以數字表示的意義。



然後，當您向 Legal Eagle 提問時，我們會使用 Firestore 的向量搜尋功能，找出與查詢最相關的法律文字區塊。RAG 會使用擷取的脈絡資訊，根據實際法律資訊提供回覆，而非僅憑 LLM 的想像力！

- 👉 在新分頁/視窗中，前往 [Google Cloud 控制台](#) 的 Firestore。
- 👉 按一下「建立資料庫」

Google Cloud

Search (/) for resources, docs, products and more

Firestore

All databases

Firestore is an enterprise-grade, serverless document database service—now with MongoDB compatibility

Easily store and read semi-structured JSON data on a differentiated Firestore database service with virtually unlimited scalability, industry-leading availability with up to 99.999% SLA, and single digit milliseconds read latency using flexible Firestore, Datastore, or MongoDB-compatible interfaces.

CREATE A FIRESTORE DATABASE

Show deleted

Database ID	↑	Edition	Configuration	Location	Creation time (UTC-4)	Scheduled backups	Encryption type	Encryption key	Actions
-------------	---	---------	---------------	----------	-----------------------	-------------------	-----------------	----------------	---------

You don't have any databases in this project yet.

CREATE DATABASE

👉 選擇 `Native mode` 和資料庫名稱做為 `(default)` 。

👉 選取單一 `region` : `us-central1` ，然後按一下「建立資料庫」。Firestore 會佈建資料庫，這可能需要幾分鐘。

Google Cloud

Search (/) for resources, docs, products and more

← Create database

Name your database

Permanent choice

Database ID * (default)

☒ Standard edition

Firestore's simple query engine with automatic indexing.

☐ Enterprise edition

PREVIEW

Firestore's advanced query engine with high customizability and MongoDB compatibility.

Not sure which edition is right for you? [Compare editions](#)

Configuration options

These presets determine how you structure and interact with your data.

☒ Firestore native

Enables Firestore's native server-side, web and mobile SDKs

Security rules

These rules provide access control and data validation for web and mobile SDKs.

☒ Restrictive

Deny all reads and writes by default

☐ Open

Allow anyone to view, edit and delete all data for the next 30 days.

☐ Firestore with Datastore compatibility

Enables Firestore's implementation of Datastore compatibility for server-side SDKs.

Location

Permanent choice. Determines where your computing resources and data are located. Affects cost, performance and replication. [Learn more](#)

Location type

☒ Region

99.99% availability SLA at general availability

☐ Multi-region

99.999% availability SLA at general availability, highest availability across the largest area

Region * us-central1 (Iowa)

SHOW ENCRYPTION OPTIONS

CREATE DATABASE

CANCEL

Pricing summary

Firestore is billed based on operations, storage and network usage¹. Location affects rates. [Learn more](#)

Operations	Price
Document reads²	\$0.03 per 100,000 documents
Document writes	\$0.09 per 100,000 documents
Document deletes	\$0.01 per 100,000 documents
Stored data	\$0.15 GiB/month

1. Network traffic costs are dependent on the location of your database and application request behaviour. [Learn more](#)

2. Aggregation queries are charged 1 document read for each batch of up to 1,000 index documents matched by the query.

HIDE DETAILS

👉 返回 Cloud IDE 的終端機，在 `embedding_vector` 欄位建立向量索引，以便在 `legal_documents` 集合中啟用向量搜尋。

```
export PROJECT_ID=$(gcloud config get project)
gcloud firestore indexes composite create \
--collection-group=legal_documents \
--query-scope=COLLECTION \
--field-config field-path=embedding,vector-config='{ "dimension": "768", "flat": "{}"}' \
--project=${PROJECT_ID}
```

Firestore 會開始建立向量索引。建立索引可能需要一些時間，尤其是較大的資料集。索引會顯示「建立中」狀態，建構完成後就會轉換為「就緒」狀態。

Google Cloud

My Project 94745

Search (/) for resources, docs, products and more

Search

Firestore

All databases > DATABASE (default)

Database

Firestore studio

Indexes

Import/Export

Disaster recovery

Time to live (TTL)

Security rules

Insights

Usage

Monitoring

Key Visualizer

Indexes

COMPOSITE

SINGLE FIELD

Composite indexes support queries that match across multiple fields by indexing combinations of fields in a collection. [Learn more](#)

Composite indexes

+ CREATE INDEX

Filter

Collection ID

Fields

Query scope

Size

Entries

Actions

↑

legal_documents

__name__: ASC

embedding: VECTOR<768>

Collection

-

-

Indexes

COMPOSITE

SINGLE FIELD

Composite indexes support queries that match across multiple fields by indexing combinations of fields in a collection. [Learn more](#)

Composite indexes

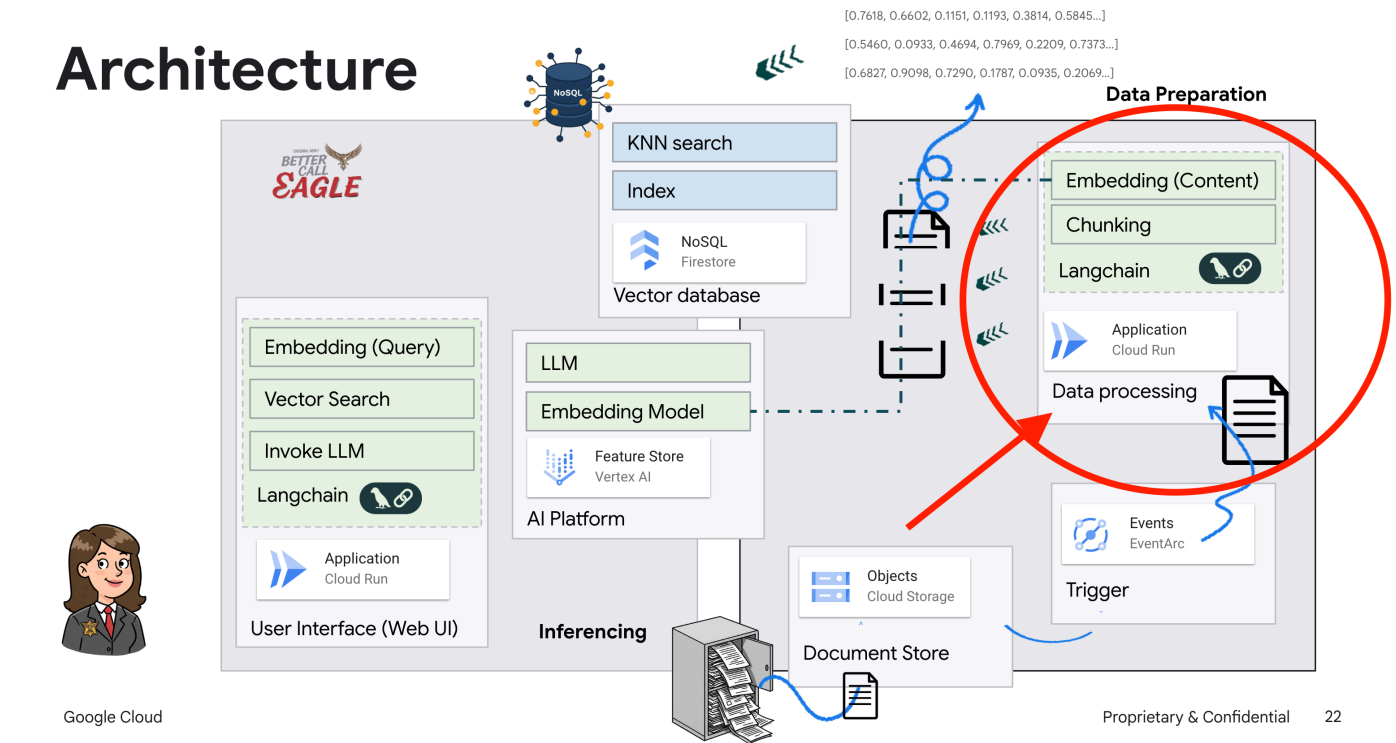
+ CREATE INDEX

Filter	Collection ID	Fields	Query scope	Size	Entries	Actions
	↑	legal_documents	__name__: ASC	embedding: VECTOR<768>	Collection	-

7. 將資料載入向量儲存區

我們已瞭解 RAG 和向量儲存區，現在要建構引擎，為法律資料庫填入內容！那麼，如何讓法律文件「可依含義搜尋」？魔法就在於嵌入項目！您可以將嵌入項目視為將字詞、句子，甚至是整份文件轉換為數值向量，也就是擷取語意的數字清單。相似概念在向量空間中會取得「接近」彼此的向量。我們會使用強大的模型 (例如 Vertex AI 的模型) 執行這項轉換。

如要自動載入文件，我們將使用 **Cloud Run functions** 和 **Eventarc**。**Cloud Run Functions** 是一種輕量型無伺服器容器，只會在需要時執行程式碼。我們會將文件處理 Python 指令碼封裝至容器，並以 Cloud Run 函式的形式部署。



👉 在新分頁/視窗中前往「Cloud Storage」。

👉 按一下左側選單中的「Buckets」。

👉 點按頂端的「+ 建立」按鈕。

👉 設定 bucket (重要設定)：

- **bucket 名稱：** 'yourprojectID'-doc-bucket (結尾必須加上 -doc-bucket 後綴)
- **區域：** 選取 `us-central1` 區域。
- **儲存空間級別：**「標準」。Standard 適合經常存取的資料。
- **存取控管：** 保留預設選取的「統一存取控管」。這可提供一致的值區層級存取控管。
- **進階選項：** 在本教學課程中，預設設定通常就足夠了。

👉 按一下「建立」按鈕，建立值區。

👉 系統可能會彈出視窗，說明禁止公開存取功能。勾選方塊，然後按一下「確認」。

現在您會在 Bucket 清單中看到新建立的 bucket。請記下 **bucket** 名稱，後續步驟會用到。

步驟 8 · 約 0 分鐘

8. 設定 Cloud Run 函式

👉 在 Cloud Shell 程式碼編輯器中，前往工作目錄 **legal-eagle**：在程式碼編輯器終端機中使用 `cd` 指令建立資料夾。

```
cd ~/legal-eagle
mkdir loader
cd loader
```

👉 建立 `main.py`、`requirements.txt` 和 `Dockerfile` 檔案。在 Cloud Shell 終端機使用 `touch` 指令建立檔案：

```
touch main.py requirements.txt Dockerfile
```

您會看到名為 `*loader` 的新資料夾和三個檔案。

👉 編輯 `loader` 資料夾下的 `main.py`。在左側的檔案總管中，前往您建立檔案的目錄，然後按兩下 `main.py`，在編輯器中開啟檔案。

將下列 Python 程式碼貼入 `main.py`：

這個應用程式會處理上傳至 GCS 值區的新檔案、將文字分割成區塊、為每個區塊產生嵌入內容，並將區塊及其嵌入內容儲存在 Firestore 中。

```

import os
import json
from google.cloud import storage
import functions_framework
from langchain_google_vertexai import VertexAI, VertexAIEmbeddings
from langchain_google_firestore import FirestoreVectorStore
from langchain.text_splitter import RecursiveCharacterTextSplitter
import vertexai

PROJECT_ID = os.environ.get("GOOGLE_CLOUD_PROJECT") # Get project ID from env
embedding_model = VertexAIEmbeddings(
    model_name="text-embedding-004" ,
    project=PROJECT_ID,)
COLLECTION_NAME = "legal_documents"
# Create a vector store
vector_store = FirestoreVectorStore(
    collection="legal_documents",
    embedding_service=embedding_model,
    content_field="original_text",
    embedding_field="embedding",
)
@functions_framework.cloud_event
def process_file(cloud_event):
    print(f"CloudEvent received: {cloud_event.data}") # Print the parsed event data

    """Triggered by a Cloud Storage event.
    Args:
        cloud_event (functions_framework.CloudEvent): The CloudEvent
            containing the Cloud Storage event data.
    """
    try:
        event_data = cloud_event.data
        bucket_name = event_data['bucket']
        file_name = event_data['name']
    except (json.JSONDecodeError, AttributeError, KeyError) as e: # Catch JSON errors
        print(f"Error decoding CloudEvent data: {e} - Data: {cloud_event.data}")
        return "Error processing event", 500 # Return an error response

    print(f"New file detected in bucket: {bucket_name}, file: {file_name}")
    storage_client = storage.Client()
    bucket = storage_client.bucket(bucket_name)
    blob = bucket.blob(file_name)
    try:
        # Download the file content as string (assuming UTF-8 encoded text file)
        file_content_string = blob.download_as_string().decode("utf-8")
        print(f"File content downloaded. Processing...")
        # Split text into chunks using RecursiveCharacterTextSplitter
        text_splitter = RecursiveCharacterTextSplitter(
            chunk_size=500,
            chunk_overlap=100,
            length_function=len,
        )
        text_chunks = text_splitter.split_text(file_content_string)

```

```
print(f"Text split into {len(text_chunks)} chunks.")

# Add the docs to the vector store
vector_store.add_texts(text_chunks)
print(f"File processing and Firestore upsert complete for file: {file_name}")
return "File processed successfully", 200 # Return success response
except Exception as e:
    print(f"Error processing file {file_name}: {e}")
```

編輯 requirements.txt 。將下列程式碼貼到檔案中：

```
Flask==2.3.3
requests==2.31.0
google-generativeai>=0.2.0
langchain
langchain_google_vertexai
langchain-community
langchain-google-firestore
google-cloud-storage
functions-framework
```

步驟 9 · 約 0 分鐘

9. 測試及建構 Cloud Run 函式

👉 我們會在虛擬環境中執行這項操作，並安裝 Cloud Run 函式所需的 Python 程式庫。

```
cd ~/legal-eagle/loader
python -m venv env
source env/bin/activate
pip install -r requirements.txt
```

👉 啟動 Cloud Run 函式的本機模擬器

```
functions-framework --target process_file --signature-type=cloudevent --source main.py
```

👉 讓上一個終端機保持執行狀態，開啟新的終端機，然後執行指令將檔案上傳至 bucket。

```
export DOC_BUCKET_NAME=$(gcloud storage buckets list --format="value(name)" | grep doc-
bucket)
gsutil cp ~/legal-eagle/court_cases/case-01.txt gs://$DOC_BUCKET_NAME/
```



👉 模擬器執行時，您可以將測試 CloudEvent 傳送至模擬器。您需要 IDE 中的另一個終端機才能執行這項操作。

```
curl -X POST -H "Content-Type: application/json" \
-d '{
  "specversion": "1.0",
  "type": "google.cloud.storage.object.v1.finalized",
  "source": "//storage.googleapis.com/$DOC_BUCKET_NAME",
  "subject": "objects/case-01.txt",
  "id": "my-event-id",
  "time": "2024-01-01T12:00:00Z",
  "data": {
    "bucket": "$DOC_BUCKET_NAME",
    "name": "case-01.txt"
  }
}' http://localhost:8080/
```

系統應會傳回「OK」。

👉 您可以在 **Firestore** 中驗證資料，前往 Google Cloud 控制台並依序點選「資料庫」和「Firestore」，然後選取「資料」分頁標籤和 `legal_documents` 集合。你會看到系統在集合中建立新文件，每份文件都代表上傳檔案中的一段文字。

Google Cloud | My Project 94745 | fire | Search

Firestore | All databases > DATABASE (default)

Database

- Firestore studio
- Indexes
- Import/Export
- Disaster recovery
- Time to live (TTL)
- Security rules

Introducing scheduled backups

Firestore now offers enhanced data protection with configurable daily or weekly backups, enabling seamless restoration. Combined with point-in-time recovery and scheduled backups, your data is safeguarded against errors and disasters.

[GIVE IT A TRY](#) [LEARN MORE](#)

PANEL VIEW | QUERY BUILDER

/ > legal_documents > BVy00Ymab9Dp0lnHq4gR

(default)	legal_documents	BVy00Ymab9Dp0lnHq4gR
+ START COLLECTION	+ ADD DOCUMENT	+ START COLLECTION
legal_documents	BVy00Ymab9Dp0lnHq4gR kGD4pEogXAP9SYv1fWSP lNePaNDFfyaiMtmQOjrS nqR3HuJmWHBB3xvlGrXm	+ ADD FIELD embedding: vector<768> ? metadata: null original_text: "Legal Arguments: The prosecution argued that Do..."

👉 在執行模擬器的終端機中，輸入 `ctrl+c` 即可結束。然後關閉第二個終端機。

👉 執行 `deactivate` 指令，即可離開虛擬環境。

```
deactivate
```

步驟 10 · 約 0 分鐘

10. 建構容器映像檔並推送至 Artifacts 存放區

👉 現在要將這個應用程式部署至雲端。在檔案總管中，按兩下 Dockerfile。問問 Gemini 生成 Dockerfile，開啟 **Gemini Code Assist** 並使用下列提示詞生成檔案。

```
In the loader folder,  
Generate a Dockerfile for a Python 3.12 Cloud Run service that uses functions-framework. It  
needs to:  
1. Use a Python 3.12 slim base image.  
2. Set the working directory to /app.  
3. Copy requirements.txt and install Python dependencies.  
4. Copy main.py.  
5. Set the command to run functions-framework, targeting the 'process_file' function on port  
8080
```

最佳做法是點選「Diff with Open File」(兩個方向相反的箭頭，並接受變更)。

The screenshot displays the Cloud Shell Editor interface. The Explorer pane on the left shows the file structure of the 'XTINALIN' project, including folders like 'legal-eagle', 'court_cases', 'loader', and files like 'main.py', 'requirements.txt', and 'webapp'. The main editor area shows a diff for 'Dockerfile U'. The diff highlights changes in a green background, with a red circle around the 'Accept' button in the diff toolbar. The diff content is as follows:

```
1-  
1+ Use a Python 3.12 slim base image  
2+ FROM python:3.12-slim  
3+  
4+ # Set the working directory to /app  
5+ WORKDIR /app  
6+  
7+ # Copy requirements.txt and install Python dependencies  
8+ COPY requirements.txt .  
9+ RUN pip install --no-cache-dir -r requirements.txt  
10+  
11+ # Copy the application code  
12+ COPY main.py .  
13+  
14+ # Set the command to run functions-framework  
15+ CMD ["functions-framework", "--target", "process_file", "--port", "8080"]  
16+
```

On the right side, the Gemini Code Assist panel is open, displaying the generated Dockerfile content and an explanation of each step. The Dockerfile content is:

```
# Use a Python 3.12 slim base image  
FROM python:3.12-slim  
  
# Set the working directory to /app  
WORKDIR /app  
  
# Copy requirements.txt and install Python dependencies  
COPY requirements.txt .  
RUN pip install --no-cache-dir -r requirements.txt  
  
# Copy the application code  
COPY main.py .  
  
# Set the command to run functions-framework  
CMD ["functions-framework", "--target", "process_file", "--port", "8080"]
```

The explanation of each step is:

- FROM python:3.12-slim:**
 - This line specifies the base image for our Docker container. We're using `python:3.12-slim`, which is a minimal image based on Debian and includes Python 3.12. The 'slim' variant is preferred for Cloud Run as it reduces the image size, leading to faster deployments and less

👉 如果您是容器新手，可以參考以下實際範例：

```
# Use a Python 3.12 slim base image
FROM python:3.12-slim
# Set the working directory to /app
WORKDIR /app
# Copy requirements.txt and install Python dependencies
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
# Copy main.py
COPY main.py .
# Set the command to run functions-framework
CMD ["functions-framework", "--target", "process_file", "--port", "8080"]
```

👉 在終端機中，建構構件存放區來儲存要建構的 Docker 映像檔。

```
gcloud artifacts repositories create my-repository \
  --repository-format=docker \
  --location=us-central1 \
  --description="My repository"
```

您應該會看到「Created repository [my-repository]」（已建立存放區 [my-repository]）。

👉 執行下列指令來建構 Docker 映像檔。

```
cd ~/legal-eagle/loader
export PROJECT_ID=$(gcloud config get project)
docker build -t gcr.io/${PROJECT_ID}/legal-eagle-loader .
```

👉 你現在要將該項目推送至登錄檔

```
export PROJECT_ID=$(gcloud config get project)
docker tag gcr.io/${PROJECT_ID}/legal-eagle-loader us-central1-
docker.pkg.dev/${PROJECT_ID}/my-repository/legal-eagle-loader
docker push us-central1-docker.pkg.dev/${PROJECT_ID}/my-repository/legal-eagle-loader
```

Docker 映像檔現在位於 *my-repository* [Artifacts Repository](#)。

Google Cloud

pokedemo-test

Search (/) for resources, docs, products and more

Artifact Registry

Repositories

Settings

Images for my-repository

DELETE

EDIT REPOSITORY

SETUP INSTRUCTIONS

us-central1-docker.pkg.dev > pokedemo-test > my-repository

Repository details

Format

Docker

Type

Standard

SHOW MORE

Filter

Enter property name or value

	Name	Connection	Created	Updated
	 legal-eagle-loader	—	Just now	Just now

11. 建立 Cloud Run 函式並設定 Eventarc 觸發條件

在深入瞭解如何部署法律文件載入器之前，請先簡要認識相關元件：**Cloud Run** 是全代管無伺服器平台，可讓您快速輕鬆地部署容器化應用程式。並省去基礎架構管理工作，讓您專心編寫及部署程式碼。

我們會將文件載入器部署為 Cloud Run 服務。接著設定 Cloud Run 函式：

👉 前往 Google Cloud 控制台中的「Cloud Run」。

👉 前往「Deploy Container」(部署容器)，然後在下拉式選單中點選「SERVICE」(服務)。

👉 設定 Cloud Run 服務：

- **容器映像檔**：按一下網址欄位中的「選取」。找出您推送至 Artifact Registry 的映像檔網址 (例如 `us-central1-docker.pkg.dev/your-project-id/my-repository/legal-eagle-loader/yourimage`)。
- **服務名稱**：`legal-eagle-loader`
- **區域**：選取 `us-central1` 區域。
- **驗證**：在本研討會中，您可以允許「允許未經驗證的叫用」。在正式環境中，您可能需要限制存取權。
- **容器、網路、安全性**：預設值。

👉 按一下「建立」。Cloud Run 會部署服務。

The screenshot displays the Google Cloud console interface for creating a new Cloud Run service. The main panel shows the 'Create service' form with the following details:

- Container image URL:** A text field with a 'SELECT' button next to it, highlighted by a red arrow.
- Configure section:**
 - Service name:** Set to 'legal-eagle-loader'.
 - Region:** Set to 'us-central1 (Iowa)'.
- Endpoint URL:** Displayed as 'https://legal-eagle-loader-400327782009.us-central1.run.app'.

On the right side, a 'Select container image' dialog is open, showing the 'ARTIFACT REGISTRY' tab. It lists the project 'named-icon-449202-s9' and the repository 'us-central1-docker.pkg.dev/named-icon-449202-s9/my-repository'. Under the 'legal-eagle-loader' tag, the image '9830f38632 latest' is selected, with a red arrow pointing to the 'SELECT' button.

如要在儲存空間 bucket 中新增檔案時自動觸發這項服務，我們將使用 **Eventarc**。您可以透過 Eventarc，將各種來源的事件轉送至服務，藉此建立事件導向架構。

← Create trigger

Trigger name *
legal-eagle-upload-trigger

Trigger type
Google sources

Event provider *
Cloud Storage

Event type *
google.cloud.storage.object.v1.finalized

SHOW DETAILS

Event data content type
application/json

Bucket *
named-icon-449202-s9-doc-bucket

BROWSE

Google Cloud Storage bucket to subscribe to

Region
us-central1 (Iowa)

Region is selected based on the selected Cloud Storage bucket



Cloud Pub/Sub needs the role **roles/iam.serviceAccountTokenCreator** granted to service account **service-400327782009@gcp-sa-pubsub.iam.gserviceaccount.com** on this project to create identity tokens. You can change this later.

GRANT

[LEARN MORE](#)

Service account
Default compute service account

Identity to be used by the created Eventarc trigger.



This trigger needs the role **roles/eventarc.eventReceiver** granted to service account **400327782009-compute@developer.gserviceaccount.com** to receive events via Google sources.

GRANT



This trigger needs the role **roles/pubsub.publisher** granted to service account **service-400327782009@gs-project-accounts.iam.gserviceaccount.com** to receive events via Cloud Storage.

GRANT

[LEARN MORE](#)

GRANT

LEARN MORE

Event destination *

Cloud Run

Select a Cloud Run service *

legal-eagle-loader

CREATE

CANCEL

請忽略「locations/me-central2」的權限遭拒 (或不存在) 訊息。權限可能需要幾分鐘才能完成同步，因此在建立觸發條件時可能會失敗，請再試幾次。

Google Cloud

Eventarc

Search

9

?

Eventarc / Triggers

Standard

Triggers

Channels

Advanced

Bus

Pipelines

Triggers

CREATE TRIGGER

DELETE

Introducing Eventarc Advanced

PREVIEW

×

Eventarc Advanced is ideal for organisations with complex eventing and messaging needs, particularly those grappling with managing numerous Pub/Sub topics, Kafka queues or other third-party messaging systems. Eventarc Advanced lets you receive, filter, transform, route and deliver messages. It provides centralised control and enhanced visibility, and you can develop event-driven apps with ease.

CREATE BUS

LEARN MORE

Filter

Filter triggers

?

☐	Name ↑	Region	Event provider	Destination platform	Destination	Event type	Event channel
☐	legal-eagle-upload-trigger	us-central1	Cloud Storage	Cloud Run	legal-eagle-loader(us-central1)	google.cloud.storage.object.v1.finalized	google

12. 將法律文件上傳至 GCS bucket

👉 將法院案件檔案上傳至 GCS bucket。請記得替換 bucket 名稱。

```
export DOC_BUCKET_NAME=$(gcloud storage buckets list --format="value(name)" | grep doc-
bucket)
gsutil cp ~/legal-eagle/court_cases/case-02.txt gs://$DOC_BUCKET_NAME/
gsutil cp ~/legal-eagle/court_cases/case-03.txt gs://$DOC_BUCKET_NAME/
gsutil cp ~/legal-eagle/court_cases/case-06.txt gs://$DOC_BUCKET_NAME/
```

監控 Cloud Run 服務記錄，依序前往「Cloud Run」-> 您的服務 `legal-eagle-loader` -> 「記錄」。檢查記錄檔，確認訊息處理成功，包括：

```
xxx
POST200130 B8.3 sAPIS-Google; (+https://developers.google.com/webmasters/APIs-Google.html)
https://legal-eagle-loader-bmngruelyta-uc.a.run.app/?__GCP_CloudEventsMode=GCS_NOTIFICATION
xxx
POST200130 B520 msAPIS-Google; (+https://developers.google.com/webmasters/APIs-Google.html)
https://legal-eagle-loader-bmngruelyta-uc.a.run.app/?__GCP_CloudEventsMode=GCS_NOTIFICATION
xxx
POST200130 B514 msAPIS-Google; (+https://developers.google.com/webmasters/APIs-Google.html)
https://legal-eagle-loader-bmngruelyta-uc.a.run.app/?__GCP_CloudEventsMode=GCS_NOTIFICATION
```

視記錄設定的速度而定，您也會在這裡看到更多詳細記錄

```
"CloudEvent received:"
"New file detected in bucket:"
"File content downloaded. Processing..."
"Text split into ... chunks."
"File processing and Firestore upsert complete..."
```

在記錄中尋找錯誤訊息，並視需要進行疑難排解。

>	2025-04-16 11:20:52.844 EDT	POST 200 130 B 8.3 s	APIS-Google; (+https://developers.google.com/webmasters/APIs-Google.html)	https://legal-eagle-loader-bmngruelyta-uc.a.run.app/?__GCP_CloudEventsMode=GCS_NOTIFICATION
>	2025-04-16 11:20:55.457 EDT	POST 200 130 B 520 ms	APIS-Google; (+https://developers.google.com/webmasters/APIs-Google.html)	https://legal-eagle-loader-bmngruelyta-uc.a.run.app/?__GCP_CloudEventsMode=GCS_NOTIFICATION
>	2025-04-16 11:20:57.952 EDT	POST 200 130 B 514 ms	APIS-Google; (+https://developers.google.com/webmasters/APIs-Google.html)	https://legal-eagle-loader-bmngruelyta-uc.a.run.app/?__GCP_CloudEventsMode=GCS_NOTIFICATION
>	2025-04-16 11:21:00.503 EDT	Default STARTUP TCP probe succeeded after 1 attempt for container 'legal-eagle-loader-1' on port 8080.		
>	2025-04-16 11:36:04.430 EDT	CloudEvent received: {'kind': 'storageObject', 'id': 'christina-2005--doc-bucket/case-02.txt/1744816852149434', 'selfLink': 'https://www.googleapis.com/storage/v1/b/christina-2005--doc-bucket/o/case-02.txt', 'name': 'case-02.txt'}		
>	2025-04-16 11:36:04.430 EDT	New file detected in bucket: christina-2005--doc-bucket, file: case-02.txt		
>	2025-04-16 11:36:04.430 EDT	File content downloaded. Processing...		
>	2025-04-16 11:36:04.430 EDT	Text split into 4 chunks.		
>	2025-04-16 11:36:04.430 EDT	CloudEvent received: {'kind': 'storageObject', 'id': 'christina-2005--doc-bucket/case-03.txt/1744816854679456', 'selfLink': 'https://www.googleapis.com/storage/v1/b/christina-2005--doc-bucket/o/case-03.txt', 'name': 'case-03.txt'}		
>	2025-04-16 11:36:04.430 EDT	New file detected in bucket: christina-2005--doc-bucket, file: case-03.txt		
>	2025-04-16 11:36:04.430 EDT	CloudEvent received: {'kind': 'storageObject', 'id': 'christina-2005--doc-bucket/case-06.txt/1744816857186345', 'selfLink': 'https://www.googleapis.com/storage/v1/b/christina-2005--doc-bucket/o/case-06.txt', 'name': 'case-06.txt'}		
>	2025-04-16 11:36:04.430 EDT	New file detected in bucket: christina-2005--doc-bucket, file: case-06.txt		
>	2025-04-16 11:36:04.430 EDT	File content downloaded. Processing...		
>	2025-04-16 11:36:04.430 EDT	Text split into 3 chunks.		
>	2025-04-16 11:36:04.430 EDT	File content downloaded. Processing...		
>	2025-04-16 11:36:04.430 EDT	Text split into 3 chunks.		
>	2025-04-16 11:36:04.430 EDT	File processing and Firestore upsert complete for file: case-02.txt		

👉 在 **Firestore** 中驗證資料。然後開啟 `legal_documents` 集合。

👉 集合中應該會顯示新建立的文件。每份文件都代表您上傳檔案中的一段文字，並包含：

metadata: currently empty
original_text_chunk: The text chunk content.
embedding_: A list of floating-point numbers (the Vertex AI embedding).

Firestore

Database

Firestore studio

Indexes

Import/Export

Disaster recovery

Time to live (TTL)

Security rules

Insights

Usage

Monitoring

Key Visualizer

All databases > DATABASE (default)

Introducing scheduled backups

Firestore now offers enhanced data protection with configurable daily or weekly backups, enabling seamless restoration. Combined with point-in-time recovery and scheduled backups, your data is safeguarded against errors and disasters.

GIVE IT A TRY

LEARN MORE

PANEL VIEW

QUERY BUILDER

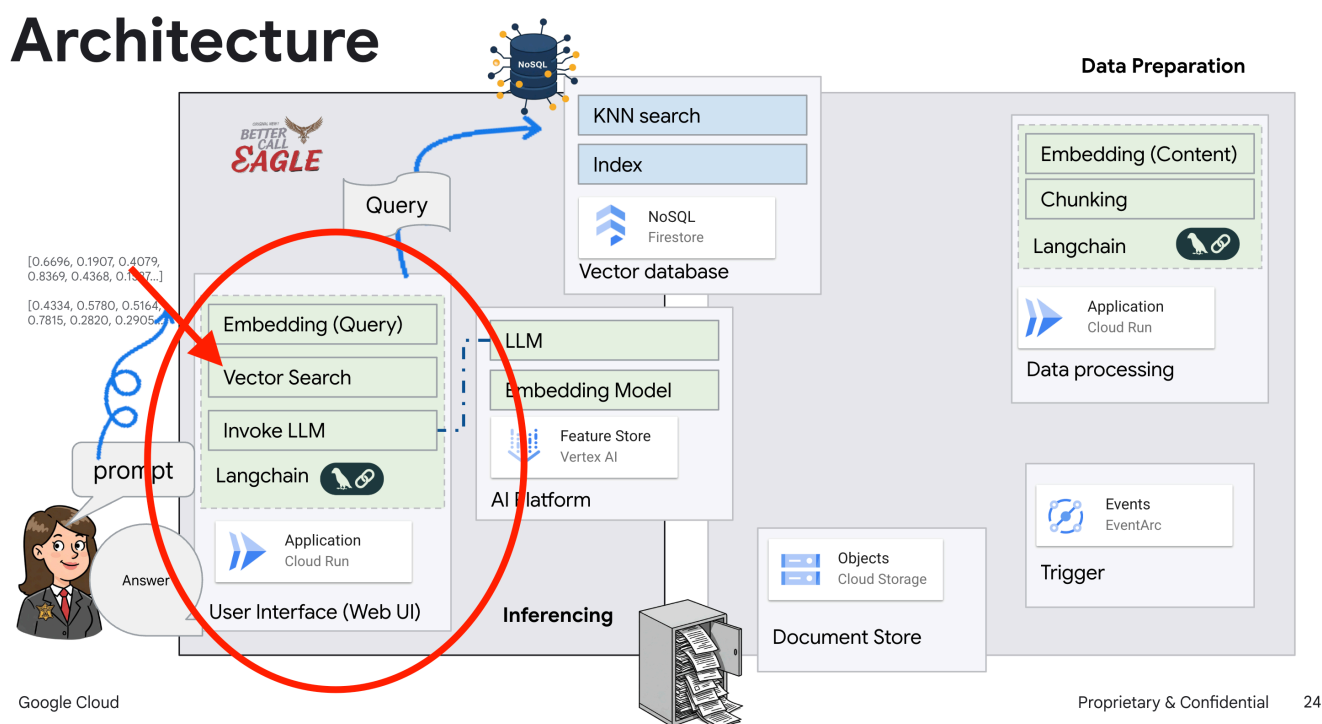
/ > legal_documents > 0MW6k2c35WWq4ZtFeHxk

(default)	legal_documents	0MW6k2c35WWq4ZtFeHxk
+ START COLLECTION	+ ADD DOCUMENT	+ START COLLECTION
legal_documents	2grOAiejCmetbhDhL09 2jNcmbxHqbJfPUGLluZt 2mHfhFVSyD1dtJij4y3G 2qZTdTTXFvsPTUgtjrx	+ ADD FIELD embedding: vector<768> metadata: null original_text: "United States v. John Doe"

13. 導入 RAG

LangChain 是一套強大的框架，可簡化大型語言模型 (LLM) 輔助應用程式的開發作業。LangChain 提供高階抽象層，可簡化大型語言模型 API、提示工程和資料處理的複雜性。並提供預先建構的元件和工具，可執行各種工作，例如連結各種 LLM (如 OpenAI、Google 或其他來源的 LLM)、建構複雜的作業鏈 (例如先檢索資料再產生摘要)，以及管理對話記憶體。

就 RAG 而言，LangChain 中的向量儲存空間是啟用 RAG 檢索功能的必要條件。這類資料庫專門用於有效率地儲存及查詢向量嵌入，並將語意相似的文字片段對應至向量空間中相近的點。LangChain 會處理低階管道，讓開發人員專注於 RAG 應用程式的核心邏輯和功能。這能大幅縮短開發時間並降低複雜度，讓您快速製作原型並部署以 RAG 為基礎的應用程式，同時運用 Google Cloud 雲端基礎架構的穩定性和可擴充性。



瞭解 LangChain 後，您現在需要更新 webapp 資料夾下的 legal.py 檔案，才能實作 RAG。這樣一來，LLM 就能在 Firestore 中搜尋相關文件，然後再提供答案。

👉 從 langchain 和 vertexai 匯入 `FirestoreVectorStore` 和其他必要模組。將下列內容新增至目前的 `legal.py`

```
from langchain_google_vertexai import VertexAIEmbeddings
from langchain_google_firestore import FirestoreVectorStore
```

👉 初始化 Vertex AI 和嵌入模型。您將使用 `text-embedding-004`。匯入模組後，請立即新增下列程式碼。

```
PROJECT_ID = os.environ.get("GOOGLE_CLOUD_PROJECT") # Get project ID from env
embedding_model = VertexAIEmbeddings(
    model_name="text-embedding-004" ,
    project=PROJECT_ID,)

```

👉 建立指向 `legal_documents` 集合的 `FirestoreVectorStore`，使用已初始化的嵌入模型，並指定內容和嵌入欄位。請在之前的嵌入模型程式碼後方新增這段程式碼。

```
COLLECTION_NAME = "legal_documents"
# Create a vector store
vector_store = FirestoreVectorStore(
    collection="legal_documents",
    embedding_service=embedding_model,
    content_field="original_text",
    embedding_field="embedding",
)
```

👉 定義名為 `search_resource` 的函式，該函式會接收查詢、使用 `vector_store.similarity_search` 執行相似度搜尋，並傳回合併結果。

```
def search_resource(query):
    results = []
    results = vector_store.similarity_search(query, k=5)

    combined_results = "\n".join([result.page_content for result in results])
    print(f"==>{combined_results}")
    return combined_results
```

👉 取代 `ask_llm` 函式，並使用 `search_resource` 函式，根據使用者的查詢擷取相關情境。

```
def ask_llm(query):
    try:
        query_message = {
            "type": "text",
            "text": query,
        }
        relevant_resource = search_resource(query)

        input_msg = HumanMessage(content=[query_message])
        prompt_template = ChatPromptTemplate.from_messages(
            [
                SystemMessage(
                    content=(
                        "You are a helpful assistant, and you are with the attorney in a courtroom, you are helping him to win the case by providing the information he needs "
                        "Don't answer if you don't know the answer, just say sorry in a funny way possible"
                        "Use high energy tone, don't use more than 100 words to answer"
                        f"Here is some context that is relevant to the question {relevant_resource} that you might use"
                    )
                ),
                input_msg,
            ]
        )
        prompt = prompt_template.format()

        response = llm.invoke(prompt)
        print(f"response: {response}")
        return response
    except Exception as e:
        print(f"Error sending message to chatbot: {e}") # Log this error too!
        return f"Unable to process your request at this time. Due to the following reason: {str(e)}"
```

👉 選用：西班牙文版

Sustituye el siguiente texto como se indica: `You are a helpful assistant, to` `You are a helpful assistant that speaks Spanish,`

👉 在 `legal.py` 中導入 RAG 後，請先在本機測試，再進行部署。如要執行應用程式，請使用下列指令：

```
cd ~/legal-eagle/webapp
source env/bin/activate
python main.py
```

👉 使用 `webpreview` 存取應用程式、與助理交談，然後輸入 `ctrl+c` 結束在本機執行的程序，並執行 `deactivate` 退出虛擬環境。

```
deactivate
```

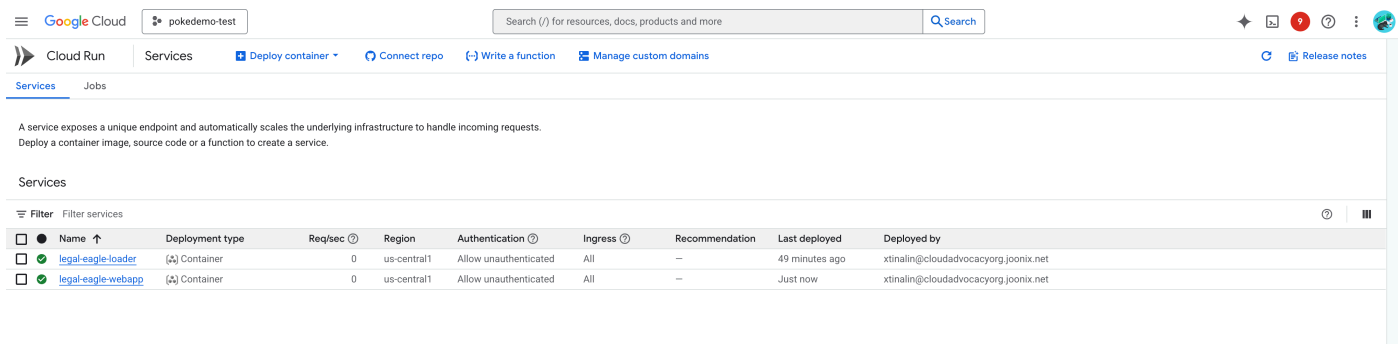
👉 如要將網頁應用程式部署至 Cloud Run，做法與載入器函式類似。您將建構 Docker 映像檔、加上標記並推送至 Artifact Registry：

```
export PROJECT_ID=$(gcloud config get project)
docker build -t gcr.io/${PROJECT_ID}/legal-eagle-webapp .
docker tag gcr.io/${PROJECT_ID}/legal-eagle-webapp us-central1-
docker.pkg.dev/${PROJECT_ID}/my-repository/legal-eagle-webapp
docker push us-central1-docker.pkg.dev/${PROJECT_ID}/my-repository/legal-eagle-webapp
```

👉 現在要將網頁應用程式部署至 Google Cloud。在終端機中執行下列指令：

```
export PROJECT_ID=$(gcloud config get project)
gcloud run deploy legal-eagle-webapp \
  --image us-central1-docker.pkg.dev/${PROJECT_ID}/my-repository/legal-eagle-webapp \
  --region us-central1 \
  --set-env-vars=GOOGLE_CLOUD_PROJECT=${PROJECT_ID} \
  --allow-unauthenticated
```

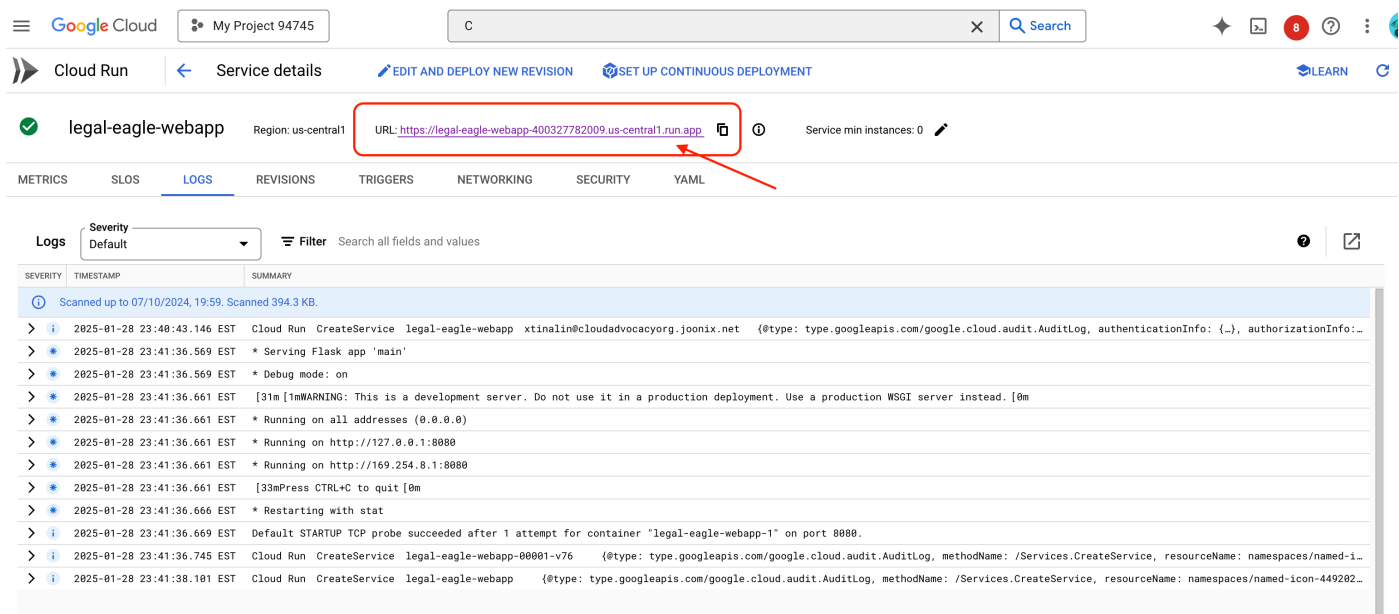
前往 Google Cloud 控制台的 [Cloud Run](#)，確認部署作業。您應該會看到名為 `legal-eagle-webapp` 的新服務列出。



The screenshot shows the Google Cloud console interface. At the top, there's a search bar and navigation tabs for 'Cloud Run', 'Services', 'Deploy container', 'Connect repo', 'Write a function', and 'Manage custom domains'. Below the navigation, there's a table of services. The 'legal-eagle-webapp' service is listed with a status of 'Running' and a deployment type of 'Container'.

Name	Deployment type	Req/sec	Region	Authentication	Ingress	Recommendation	Last deployed	Deployed by
legal-eagle-loader	(A) Container	0	us-central1	Allow unauthenticated	All	—	49 minutes ago	xtinalin@cloudadvocacyorg.joonix.net
legal-eagle-webapp	(A) Container	0	us-central1	Allow unauthenticated	All	—	Just now	xtinalin@cloudadvocacyorg.joonix.net

按一下服務前往詳細資料頁面，您可以在頂端找到可用的已部署網址。



The screenshot shows the 'Service details' page for 'legal-eagle-webapp'. The 'URL' field is highlighted with a red box and an arrow pointing to the 'LOGS' tab. The 'LOGS' tab is active, showing a list of logs with columns for 'SEVERITY', 'TIMESTAMP', and 'SUMMARY'.

SEVERITY	TIMESTAMP	SUMMARY
Info	2025-01-28 23:40:43.146 EST	Cloud Run CreateService legal-eagle-webapp xtinalin@cloudadvocacyorg.joonix.net {@type: type.googleapis.com/google.cloud.audit.AuditLog, authenticationInfo: {...}, authorizationInfo: ...}
Info	2025-01-28 23:41:36.569 EST	* Serving Flask app 'main'
Info	2025-01-28 23:41:36.569 EST	* Debug mode: on
Warning	2025-01-28 23:41:36.661 EST	[31m [1mWARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead. [0m
Info	2025-01-28 23:41:36.661 EST	* Running on all addresses (0.0.0.0)
Info	2025-01-28 23:41:36.661 EST	* Running on http://127.0.0.1:8080
Info	2025-01-28 23:41:36.661 EST	* Running on http://169.254.8.1:8080
Info	2025-01-28 23:41:36.661 EST	[33mPress CTRL+C to quit [0m
Info	2025-01-28 23:41:36.666 EST	* Restarting with stat
Info	2025-01-28 23:41:36.669 EST	Default STARTUP TCP probe succeeded after 1 attempt for container "legal-eagle-webapp-1" on port 8080.
Info	2025-01-28 23:41:36.745 EST	Cloud Run CreateService legal-eagle-webapp-00001-v76 {@type: type.googleapis.com/google.cloud.audit.AuditLog, methodName: /Services.CreateService, resourceName: namespaces/named-i-...
Info	2025-01-28 23:41:38.101 EST	Cloud Run CreateService legal-eagle-webapp {@type: type.googleapis.com/google.cloud.audit.AuditLog, methodName: /Services.CreateService, resourceName: namespaces/named-icon-449202-...

👉 現在，請在新瀏覽器分頁中開啟已部署的網址。你可以與法律助理互動，並詢問與已載入案件相關的問題(位於 `court_cases` 資料夾中)：

- Michael Brown 被判處幾年有期徒刑？
- 陳小珍的動作導致多少未授權費用？
- 在 Emily White 案件的調查中，鄰居的證詞扮演什麼角色？

👉 選用：西班牙文版

- ¿A cuántos años de prisión fue sentenciado Michael Brown?
- ¿Cuánto dinero en cargos no autorizados se generó como resultado de las acciones de Jane Smith?
- ¿Qué papel jugaron los testimonios de los vecinos en la investigación del caso de Emily White?

你會發現回覆內容現在更準確，且以你上傳的法律文件內容為依據，展現 RAG 的強大功能！

恭喜您完成研討會！您已成功使用 LLM、LangChain 和 Google Cloud 建構及部署法律文件分析應用程式。您已瞭解如何擷取及處理法律文件、使用 RAG 透過相關資訊增強 LLM 回覆，以及將應用程式部署為無伺服器服務。您將運用這些知識和建構的應用程式，進一步探索 LLM 在法律工作方面的強大功能。做得好！」

步驟 14 · 約 0 分鐘

14. 挑戰

多種媒體類型：

如何擷取及處理各種媒體類型，例如法院影片和錄音，並擷取相關文字。

線上資產：

如何即時處理網頁等線上資產。
